POLITECNICO
MILANO 1863

SCUOLA DI INGEGNERIA INDUSTRIALE
E DELL'INFORMAZIONE

MATRIX-FREE SOLVER

NUMERICAL METHODS FOR PARTIAL DIFFERENTIAL EQUATIONS
HIGH PERFORMANCE COMPUTING ENGINEERING

Daniele Confalonieri
Cristiano Corona
Giorgio Mantoan
Dmitrii Meshcheriakov

Course Instructors:
Prof. Alfio Maria Quarteroni,
Prof. Michele Bucelli

Academic year:
2025-2026

Key-words: PDE-Advection-Diffusion-Reaction, Matrix-Free, Preconditioner, Weak Formulation, Sum Factorization, Lifting, Weak Scalability, Strong Scalability

1. Introduction

A matrix-free method is a numerical approach that does not explicitly store the matrix elements, applied here to a second-order Advection-Diffusion-Reaction (ADR) equation on a hypercube.
The major motivation for matrix-free methods is the fact that on modern processors, access to main memory has become the primary bottleneck. For objects that do not fit in the caches, CPUs spend far more time waiting for data to arrive from memory than executing the actual floating-point operations. Thus, by substituting the memory lookup of matrix elements with the on-the-fly recomputation of the operator's action, we can significantly reduce overall run-time, even if this requires a substantially higher number of FLOPS.
The use of high-order polynomials in standard matrix-based solvers implies facing both memory and performance challenges, mostly related to the increased density of the discretized problem. In this context, a high-order matrix-free numerical solver proves highly advantageous. Since there is no need to assemble any global matrix, all the floating-point operations are associated directly with matrix-vector products, which represent the most demanding computational kernels at each iteration of the solver.
To ensure this approach truly gains in performance, the solver relies on two key high-performance computing strategies: vectorization and sum-factorization. **Vectorization** enables algebraic operations on multiple mesh cells at the same time. Concurrently, **sum-factorization** reduces the operational complexity of tensor-product evaluations. Thanks to these techniques, the higher the polynomial degree, the higher the computational advantages provided by the matrix-free solver. Moreover, the small memory footprint required by the matrix-free implementation allows for its exploitation in massively parallel, large-scale architectures.
Finally, efficiency is further boosted by implementing a matrix-free Geometric Multigrid (GMG) preconditioner. Ultimately, this implementation shifts the computational profile: while in a matrix-based approach the assem-

bling and solving phases can dominate the execution time, the matrix-free formulation significantly lowers these costs, providing substantial speed-ups and more efficient hardware utilization.

2. Problem Equation

We consider the steady Advection-Diffusion-Reaction (ADR) equation. Let $\Omega \subset R^n$ be an open, bounded domain with a sufficiently smooth boundary $\partial\Omega$. We assume that the boundary is partitioned into two disjoint measurable sets, the Dirichlet boundary Γ_D and the Neumann boundary Γ_N, such that $\Gamma_D \cup \Gamma_N = \partial\Omega$ and $\Gamma_D \cap \Gamma_N = \emptyset$.
The boundary value problem is formulated as follows:

$$\begin{cases} -\nabla \cdot (\mu \nabla u) + \nabla \cdot (\boldsymbol{\beta} u) + \gamma u = f & \text{in } \Omega & \text{(1a)} \\ u = g & \text{on } \Gamma_D & \text{(1b)} \\ \mu \nabla u \cdot \mathbf{n} = h & \text{on } \Gamma_N & \text{(1c)} \end{cases}$$

where $u(\boldsymbol{x})$ is the unknown scalar field, and $\boldsymbol{n}$ denotes the outward unit normal vector to $\partial\Omega$.
The strictly positive function $\mu(\boldsymbol{x}) : \Omega \to R$ represents the diffusion coefficient, $\boldsymbol{\beta}(\boldsymbol{x}) : \Omega \to R^n$ is the advective velocity field, and $\gamma(\boldsymbol{x}) : \Omega \to R$ is the reaction coefficient. Furthermore, f represents the forcing term, while g and h prescribe the Dirichlet and Neumann boundary data, respectively.

3. Weak Formulation

3.1. Derivation of the Weak Formulation

The transition from the strong form to the weak (variational) form is a fundamental step for the Finite Element Method (FEM) implementation. This process reduces the continuity requirements on the solution u and incorporates the boundary conditions.

3.2. Step 1: Multiplication by a Test Function

Let V be the Sobolev space $H^1(\Omega)$ and $V_0 = \{v \in V : v|_{\Gamma_D} = 0\}$ be the space of test functions that vanish on the Dirichlet boundary. We multiply the governing equation by a test function $v \in V_0$ and integrate over the entire domain Ω:

$$\int_\Omega [-\nabla \cdot (\mu \nabla u) + \nabla \cdot (\boldsymbol{\beta} u) + \gamma u]\, v \, \boldsymbol{dx} = \int_\Omega f v \, \boldsymbol{dx} \tag{2}$$

By linearity of the integral, we can separate the terms:

$$\int_\Omega -\nabla \cdot (\mu \nabla u) v \, \boldsymbol{dx} + \int_\Omega \nabla \cdot (\boldsymbol{\beta} u) v \, \boldsymbol{dx} + \int_\Omega \gamma u v \, \boldsymbol{dx} = \int_\Omega f v \, \boldsymbol{dx} \tag{3}$$

3.3. Step 2: Integration by Parts (Green's First Identity)

To lower the order of the derivatives on u, we apply the integration by parts formula (derived from the Divergence Theorem) to the diffusion term. Given the identity $\nabla \cdot (\mathbf{a}\phi) = \phi \nabla \cdot \mathbf{a} + \mathbf{a} \cdot \nabla \phi$, we have:

$$-\int_\Omega \nabla \cdot (\mu \nabla u) v \, \boldsymbol{dx} = \int_\Omega \mu \nabla u \cdot \nabla v \, \boldsymbol{dx} - \int_{\partial\Omega} v (\mu \nabla u \cdot \mathbf{n}) \, d\sigma \tag{4}$$

The boundary integral over $\partial\Omega$ is split into the Dirichlet (Γ_D) and Neumann (Γ_N) parts:

- On Γ_D: $v = 0$ by the definition of the test space V_0.
- On Γ_N: $\mu \nabla u \cdot \mathbf{n} = h$ from the Neumann boundary condition.

Thus, the boundary term simplifies to:

$$-\int_{\Gamma_N} h v \, d\sigma \tag{5}$$

3.4. Step 3: Final Form

Combining all the terms, the weak formulation of the Advection-Diffusion-Reaction problem is:
Find $u \in V$ with $u|_{\Gamma_D} = g$ such that $\forall v \in V_0$:

$$\underbrace{\int_\Omega \mu \nabla u \cdot \nabla v \, \boldsymbol{dx}}_{\text{Diffusion}} + \underbrace{\int_\Omega v \nabla \cdot (\boldsymbol{\beta} u) \, \boldsymbol{dx}}_{\text{Advection}} + \underbrace{\int_\Omega \gamma u v \, \boldsymbol{dx}}_{\text{Reaction}} = \underbrace{\int_\Omega f v \, \boldsymbol{dx} + \int_{\Gamma_N} h v \, d\sigma}_{\text{Source \& Neumann}} \tag{6}$$

3.5. Step 4: Suitable Form for the Matrix-Free Method

The advection term in the weak form can be expanded to distinguish between the transport along streamlines and the effect of the velocity field divergence. Using the product rule for the divergence operator $\nabla\cdot$:

$$\nabla \cdot (\boldsymbol{\beta} u) = \boldsymbol{\beta} \cdot \nabla u + u(\nabla \cdot \boldsymbol{\beta}) \tag{7}$$

Substituting this into the advection integral, we obtain the split into two distinct integrals:

$$\underbrace{\int_\Omega v(\boldsymbol{\beta} \cdot \nabla u) \, \boldsymbol{dx}}_{\text{Convective transport}} + \underbrace{\int_\Omega v u (\nabla \cdot \boldsymbol{\beta}) \, \boldsymbol{dx}}_{\text{Divergence contribution}} \tag{8}$$

By substituting the expanded advection term into the variational form, we obtain:

$$\int_\Omega \mu \nabla u \cdot \nabla v \, \boldsymbol{dx} + \int_\Omega v(\boldsymbol{\beta} \cdot \nabla u) \, \boldsymbol{dx} + \int_\Omega v u (\nabla \cdot \boldsymbol{\beta}) \, \boldsymbol{dx} + \int_\Omega \gamma u v \, \boldsymbol{dx} = \int_\Omega f v \, \boldsymbol{dx} + \int_{\Gamma_N} h v \, d\sigma \tag{9}$$

Since both the divergence of the velocity field and the reaction coefficient γ act as zeroth-order operators on u, they can be grouped together:

$$\int_\Omega \mu \nabla u \cdot \nabla v \, \boldsymbol{dx} + \int_\Omega v(\boldsymbol{\beta} \cdot \nabla u) \, \boldsymbol{dx} + \int_\Omega (\gamma + \nabla \cdot \boldsymbol{\beta}) u v \, \boldsymbol{dx} = \int_\Omega f v \, \boldsymbol{dx} + \int_{\Gamma_N} h v \, d\sigma \tag{10}$$

4. Sum-Factorization

4.1. Limits of Matrix-based approach

In the classical finite element approach, the global operator A is explicitly constructed by iterating over each cell K to compute local stiffness matrices A_K, which are then scattered into a sparse storage format.
This approach encounters significant bottlenecks particularly for high-order methods due to the explicit storage of A is inherently memory-bound: as the polynomial degree p increases, the size of local dense matrices scales as $\mathcal{O}(p^{2d})$. This results in prohibitive memory requirements and low arithmetic intensity.

4.2. The Matrix-Free Approach via Sum-Factorization

To overcome the "memory wall", a matrix-free approach computes the action of the operator “on-the-fly” at each solver iteration without forming A_K. This efficiency relies on **Sum-Factorization**, a technique exploiting the tensor-product structure of basis functions on hypercubic elements.
On a d-dimensional reference cell $\hat{K} = [0,1]^d$, a multivariate basis function $\Phi_i(x)$ is defined as the product of 1D Lagrange polynomials $\{\varphi_j\}_{j=0}^p$:

$$\Phi_i(x_1, \ldots, x_d) = \prod_{k=1}^{d} \varphi_{i_k}(x_k) \tag{11}$$

This tensor structure is preserved by the gradient operator. The partial derivative with respect to the m-th coordinate is expressed as:

$$\frac{\partial \Phi_i}{\partial x_m} = \frac{d\varphi_{i_m}(x_m)}{dx_m} \prod_{k \neq m} \varphi_{i_k}(x_k) \tag{12}$$

4.3. Algorithm (3D Case)

Directly evaluating the solution u at a quadrature point $x_{q,r,s} = (\xi_q, \eta_r, \zeta_s)$ for a 3D element involves computing a triple sum:

$$u(\xi_q, \eta_r, \zeta_s) = \sum_{i=0}^{p} \sum_{j=0}^{p} \sum_{k=0}^{p} u_{i,j,k} \Phi_i(\xi_q, \eta_r, \zeta_s) \tag{13}$$

By exploiting the tensor-structure, this can be rewritten as:

$$u(\xi_q, \eta_r, \zeta_s) = \sum_{i=0}^{p} \sum_{j=0}^{p} \sum_{k=0}^{p} u_{i,j,k} \varphi_i(\xi_q) \varphi_j(\eta_r) \varphi_k(\zeta_s) \tag{14}$$

A direct evaluation of this triple sum for all $(p+1)^3$ quadrature points would require $\mathcal{O}(p^6)$ operations. Sum-Factorization decouples the multidimensional sum into a sequence of nested univariate contractions:

1. **Contraction along ζ:**

$$v_{i,j,s} = \sum_{k=0}^{p} u_{i,j,k} \varphi_k(\zeta_s) \tag{15}$$

2. **Contraction along η:**

$$w_{i,r,s} = \sum_{j=0}^{p} v_{i,j,s} \varphi_j(\eta_r) \tag{16}$$

3. **Contraction along ξ:**

$$u(\xi_q, \eta_r, \zeta_s) = \sum_{i=0}^{p} w_{i,r,s} \varphi_i(\xi_q) \tag{17}$$

Each step corresponds to a dense matrix-matrix multiplication, which can be easily vectorized using SIMD instructions (e.g., AVX-512) on modern CPUs.

Asymptotic Scaling and Theoretical Speedup The transition from matrix-based to matrix-free evaluation represents a shift in fundamental scaling:

- **Standard Matrix-Based:** Applying a local operator involves a dense matrix-vector product. With $n = (p+1)^d$ DoFs per cell, complexity scales as $\mathcal{O}((p+1)^{2d})$.
- **Sum-Factorization:** The process consists of d steps, each costing $(p+1)^2$. The total complexity scales as $\mathcal{O}(d \cdot (p+1)^{d+1})$.

For a 3D problem ($d = 3$), the theoretical speedup is summarized below:

Polynomial Degree p	Matrix-Based $(p+1)^6$	Sum-Factorization $3(p+1)^4$	Theoretical Speedup
2	729	243	3.00x
3	4,096	768	5.33x
4	15,625	1,875	8.33x
5	46,656	3,888	12.00x
6	117,649	7,203	16.33x

Table 1: Comparison of operations count and theoretical speedup per cell for a 3D problem ($d = 3$).

5. The `deal.II` Matrix-Free Infrastructure

5.1. From `FEValues` to `FEEvaluation`

In a standard matrix-based `deal.II` application, the assembly loop relies on the `FEValues` class to compute shape function values and gradients at the quadrature points. However, `FEValues` evaluates these quantities one cell and one quadrature point at a time, storing the results in dense data structures. This approach is intrinsically scalar, memory-bound, and completely unsuited for on-the-fly operator evaluation.

To achieve the theoretical speedups of sum-factorization, our implementation leverages the `MatrixFree` framework and the `FEEvaluation` class. Unlike its matrix-based counterpart, `FEEvaluation` is heavily optimized for

SIMD vectorization. It processes batches of cells simultaneously, automatically padding the data to fill the wide vector registers of modern CPUs (e.g., 512-bit AVX-512). Furthermore, it internally implements the 1D tensor contractions required by the sum-factorization, completely hiding the complexity of the nested loops described in Section 4 and maximizing the arithmetic intensity of the execution.

5.2. The Custom `ADROperator`

Because the global stiffness matrix A is never explicitly assembled, the linear solvers and the Multigrid smoothers require a way to compute the matrix-vector product $v = Au$ strictly on-the-fly. To achieve this, we designed a custom `ADROperator` class that encapsulates the physics of the ADR problem.
The core of the operator resides in its `local_apply` function, which is used into the highly parallel `MatrixFree::cell_loop`. Within this localized kernel, `FEEvaluation` performs three main steps:

1. It gathers the degrees of freedom from the global source vector and evaluates the solution values and gradients at the quadrature points.
2. It applies the PDE coefficients (μ, $\boldsymbol{\beta}$, γ) point-wise, performing the arithmetic operations defined by the weak formulation.
3. It integrates the result and scatters the contributions back into the global destination vector.

By isolating the continuous PDE physics inside this specialized object, the code achieves a clean separation of concerns. The solvers interact with the `ADROperator` simply as an abstract linear operator, while the underlying implementation maintains maximum SIMD throughput.

5.3. Coefficient Pre-computation and SIMD Alignment

A fundamental architectural trade-off in our implementation concerns the evaluation of the spatially varying PDE coefficients (μ, $\boldsymbol{\beta}$, γ). In a strictly "pure" matrix-free approach, these coefficients would be evaluated continuously on-the-fly at the quadrature points during every application of the operator. However, repeatedly calling complex analytical and transcendental functions within the iterative solver loop introduces a massive computational bottleneck, severely degrading the instruction throughput.
To circumvent this, we opted for an pre-computation strategy. As implemented in the `ADROperator::evaluate_coefficients` routine, the continuous functions are sampled exclusively once during the `setup_system` phase. By doing so, we deliberately trade a small fraction of the saved RAM footprint to securely cache the physical properties of the domain.
More importantly, the pre-computed values are not stored in standard arrays, but are explicitly packed into `deal.II`'s `VectorizedArray` data structures. This manual SIMD lane unrolling guarantees that, during the intensive `local_apply` execution, the PDE coefficients are fetched from the L1 cache perfectly aligned with the vector registers of the CPU. Consequently, what would mathematically be a computationally expensive function evaluation is reduced to a contiguous memory load, perfectly complementing the sum-factorization throughput.

6. Preconditioning

It is well known from Numerical Analysis that preconditioners are used for increasing the speed of convergence when solving linear systems. Unfortunately, most of the more frequently used preconditioners such as SSOR, ILU or algebraic multigrid (AMG) cannot be used here because their implementation requires knowledge of the elements of the system matrix and of the sparsity pattern.
The simplest preconditioner to improve convergence properties is the Jacobi preconditioner. It is ideal for matrix-free implementations because it requires only the diagonal values of the matrix, which can be computed using the `compute_diagonal` method of the `ADROperator` class. However, the Jacobi preconditioner exhibits limited effectiveness; therefore, an alternative option is necessary to achieve better efficiency and utilize fewer resources.
One solution is to use geometric multigrid methods. They are very fast, and they are suitable for our purpose since all ingredients, including the transfer between different grid levels, can be expressed in terms of matrix-vector products related to a collection of cells. All one needs to do is to find a smoother that is based on matrix-vector products rather than all the matrix entries. One such candidate would be a damped Jacobi iteration that requires access to the matrix diagonal, but it is often not sufficiently good in damping all high-frequency errors. The properties of the Jacobi method can be improved by transitioning to the so-called Chebyshev iterations. The Chebyshev iteration is described by a polynomial expression of the matrix-vector product where the coefficients ω_i can be chosen to achieve certain properties, in this case to smooth the high-frequency components of the error which are associated to the eigenvalues of the Jacobi-preconditioned matrix. At degree zero, the Jacobi

method with optimal damping parameter is retrieved, whereas higher order corrections are used to improve the smoothing properties.
In our code, to achieve high performance through massive parallelization, we use MPI utilities implemented in `deal.II`'s classes and methods. The `deal.II` parallel vector class, `LinearAlgebra::distributed::Vector`, holds the processor-local part of the solution as well as data fields for *ghosted DoFs*, i.e., DoFs that are owned by a remote processor but accessed by cells that are owned by the present processor; *locally active degrees of freedom*. In the storage scheme of the vector, the data array extends beyond the processor-local part of the solution with further vector entries available for the ghosted degrees of freedom. This gives a contiguous index range for all locally active degrees of freedom.
Since matrix-free operations can be thought of doing linear algebra that is performance-critical, and performance-critical code cannot waste time on doing MPI-global to MPI-local index translations, the availability of an index space local to one MPI rank is fundamental. The way things are accessed here is a direct array access. This is provided through `LinearAlgebra::distributed::Vector::local_element()`, but it is actually rarely needed because all of this happens internally in `FEEvaluation`.
Building upon this parallel data structure, the geometric multigrid preconditioner is implemented within our `MatrixFreeSolverMG` class. The setup requires a hierarchy of operators for each level of the locally refined mesh.
To achieve this, the triangulation is constructed with the `Triangulation::construct_multigrid_hierarchy` flag. During the `setup_system()` phase, degrees of freedom are distributed across all multigrid levels, and a separate `MatrixFree` operator is instantiated for each grid level, stored in an `mg_matrices` array.
A critical feature of the implementation is the dynamic selection of smoothers and solvers based on the physical properties of the PDE.

```
bool is_advection_zero = false;

    if (std::dynamic_pointer_cast<const
    Functions::ZeroFunction<dim, NumberType>>(beta_function))
    {
        is_advection_zero = true;
    }
    else if(auto const_func = std::dynamic_pointer_cast<const
    Functions::ConstantFunction<dim, NumberType>>(beta_function))
    {
        is_advection_zero = true;
        for (unsigned int d = 0; d < dim; ++d) {
            if (std::abs(const_func->value(Point<dim>(), d)) > 1e-12) {
                is_advection_zero = false;
                break;
            }
        }
    }
```

If the problem is purely diffusive/reactive (symmetric), the solver utilizes the Chebyshev smoother discussed previously and a Conjugate Gradient (CG) outer solver; otherwise, the solver utilizes the default settings with damped Jacobi as the smoother and GMRES as the outer solver. The coarse grid problem is tackled by heavily iterating the Chebyshev polynomial.
Conversely, when solving non-symmetric advection-diffusion systems, the standard Chebyshev properties degrade. To address this, the implementation re-purposes the Chebyshev object to emulate a damped Jacobi smoother by restricting the polynomial degree to 1 and fixing the maximum eigenvalue to $1/\omega$ (using a damping factor $\omega = 0.6$). Furthermore, an iterative GMRES solver handles the coarse grid solution:
Finally, grid transfers are seamlessly handled matrix-free by `MGTransferMatrixFree`. All components—the matrix hierarchy, coarse solver, smoother, and transfer operator—are wrapped into a `Multigrid` object and subsequently passed to the `PreconditionMG` wrapper. This directly feeds into the Krylov subspace solvers, resulting in drastically reduced memory bandwidth requirements and excellent parallel efficiency.

7. Handling of Inhomogeneous Dirichlet Boundary Conditions

The imposition of inhomogeneous Dirichlet boundary conditions requires special care in a matrix-free framework. Let N be the set of all degrees of freedom (DoFs) and $N_D \subset N$ be the subset of DoFs located on the Dirichlet boundary Γ_D. The discrete solution u_h can be decomposed into an unknown interior part and a known boundary

part:

$$u_h(\mathbf{x}) = \sum_{j \in N \setminus N_D} U_j \varphi_j(\mathbf{x}) + \sum_{j \in N_D} g_j \varphi_j(\mathbf{x})$$

where g_j are the nodal values interpolating the boundary function $g(\mathbf{x})$. This corresponds to constructing a discrete lifting u_g of the Dirichlet data and solving for the non homogeneous correction $\tilde{u}_h$. Inserting this decomposition into the general weak form $a(\varphi_i, u_h) = f(\varphi_i)$ and isolating the unknown coefficients yields the modified linear system:

$$\sum_{j \in N \setminus N_D} a(\varphi_i, \varphi_j) U_j = f(\varphi_i) - \sum_{j \in N_D} a(\varphi_i, \varphi_j) g_j \quad \forall i \in N \setminus N_D$$

The second term on the right-hand side represents the discrete shift required to account for the non-zero boundary values.

7.1. The Matrix-Based Approach

In a traditional matrix-based framework, this shift is handled transparently at the algebraic level. Once the global stiffness matrix A and the right-hand side vector F are assembled without considering the constraints, `deal.II` utilities such as `AffineConstraints::distribute_local_to_global()` manipulate the algebraic structures directly. Specifically, the rows corresponding to $i \in N_D$ are zeroed out, a 1.0 is placed on the diagonal, and the off-diagonal entries A_{ij} are used to subtract the boundary contributions from the corresponding entries in the right-hand side vector.

7.2. The Matrix-Free Challenge

By definition, matrix-free methods avoid the explicit storage of the global matrix A, making the approach described above unfeasible. The operator action is evaluated on the fly when needed. Since the iterative solver expects a linear operator, it implicitly solves the homogeneous problem (i.e., subject to zero Dirichlet boundary conditions). Consequently, the right-hand side vector must be explicitly constructed to include the boundary shift before the iterative solver is invoked.
While the `deal.II` documentation suggests handling this by exploiting the heavily vectorized `FEEvaluation` class, either by bypassing constraint resolution via `read_dof_values_plain()` or by maintaining multiple `AffineConstraints` objects, these approaches introduce significant architectural overhead and code complexity.

7.3. Our Hybrid Explicit Integration Approach

To balance computational efficiency and code maintainability, we opted for a hybrid approach. Since the right-hand side is assembled strictly once during the setup phase, heavily vectorizing this specific operation yields a negligible contribution compared to the iterative matrix-vector product. Therefore, we evaluate the boundary shift explicitly using the standard `FEValues` infrastructure during the right-hand side assembly.
First, an auxiliary distributed vector `u_dirichlet` is created, containing the exact values g_j on the boundary nodes and zeros elsewhere. During the local cell assembly, if a cell touches the Dirichlet boundary, we explicitly interpolate the local boundary function u_g and its gradient ∇u_g at the quadrature points:

```
bool needs_shift = false;
for (unsigned int i = 0; i < dofs_per_cell; ++i) {
    local_dirichlet(i) = u_dirichlet(local_dof_indices[i]);
    if (std::abs(local_dirichlet(i)) > 1e-14)
        needs_shift = true;
}
```

Subsequently, the exact continuous shift corresponding to $\int_{\Omega_e} (\mu \nabla u_g \cdot \nabla \phi_i + \beta \cdot \nabla u_g \phi_i + \gamma u_g \phi_i)\, d\Omega$ is evaluated and subtracted from the local forcing term:

```
if (needs_shift) {
    NumberType shift = mu_values[q] * (grad_u_g * fe_values.shape_grad(i, q)) +
                       (beta_values[q] * grad_u_g) * fe_values.shape_value(i, q) +
                       gamma_values[q] * u_g * fe_values.shape_value(i, q);
    rhs_val -= shift;
}
```

This approach computes the shift without requiring multiple constraint objects. Finally, to solve the resulting homogeneous system, the initial guess vector is zeroed out on the boundaries using `constraints.set_zero(solution)`. Once the CG/GMRES solver converges to the interior solution, the correct boundary values are restored via `constraints.distribute(solution)`.
This design ensures that the most computationally intensive phase (i.e., the operator evaluation within the iterative solver) remains fully matrix-free, SIMD-optimized, and free of boundary-handling branches, preserving the maximum theoretical throughput while keeping the RHS assembly highly readable and robust by paying a small assembly cost.

8. Software verification

8.1. Importance of verification

Verifying numerical software is a critical step to ensure that the mathematical model is correctly translated into machine instructions. This is particularly crucial in the context of a **Matrix-Free** solver. Unlike standard matrix-based approaches, where the global stiffness matrix A is explicitly assembled and stored in memory, a matrix-free method computes the operator action "on-the-fly". Consequently, it is impossible to inspect the algebraic properties of the system (such as the matrix condition number or sparsity pattern) to detect implementation errors. The operator exists only as a function, and its correctness can only be verified by analyzing the convergence of the final solution.
The intricate nested loops and SIMD instructions required to perform these contractions introduce a high risk of subtle indexing errors that might not cause runtime failures but would compromise the physical accuracy of the simulation.
To overcome the inability to inspect the algebraic system, we perform the software verification through the **Method of Manufactured Solutions (MMS)**. This technique consists of defining a known, smooth analytical solution u_{ex} over the computational domain. It is crucial to select a transcendental function (or a polynomial of sufficiently high degree) that strictly does not belong to the discrete finite element space V_h; this ensures that the discretization error is not trivially zero, preventing the verification from degenerating into a simple patch test.
By substituting u_{ex} into the PDE, we analytically derive the corresponding forcing term f and the appropriate non-homogeneous boundary conditions. These synthesized data are then fed into the numerical solver. Ultimately, the correctness of the entire implementation can be certified both quantitatively, by computing the discrete error norms between the numerical solution u_h and the manufactured u_{ex} and verifying that they decrease at the optimal theoretical rates upon mesh refinement, and qualitatively by comparing the solutions plotted by ParaView.

8.2. First problem: homogeneous boundary conditions

Let us consider a standard ADR problem of the following form:

$$\begin{cases} -\nabla \cdot (\mu \nabla u) + \nabla \cdot (\boldsymbol{\beta} u) + \gamma u = f & \text{in } \Omega, \quad (18a) \\ u = 0 & \text{on } \partial\Omega. \quad (18b) \end{cases}$$

This is a full Dirichlet problem, where:

$$\begin{aligned} \Omega &= [0,1]^3 \subset R^3 \\ \mu &= 1, \quad \boldsymbol{\beta} = (1,1,1)^T, \quad \gamma = 1 \\ u_{ex}(x,y,z) &= \sin(\pi x)\sin(\pi y)\sin(\pi z) \\ f(x,y,z) &= (3\pi^2 + 1)\sin(\pi x)\sin(\pi y)\sin(\pi z) \\ &\quad + \pi\Big[\cos(\pi x)\sin(\pi y)\sin(\pi z) \\ &\qquad + \sin(\pi x)\cos(\pi y)\sin(\pi z) \\ &\qquad + \sin(\pi x)\sin(\pi y)\cos(\pi z)\Big] \end{aligned}$$

8.2.1 Manufactured forcing function derivation

To apply the Method of Manufactured Solutions, we analytically derived the forcing term f as shown in this section.
Given the constant coefficients $\mu = 1$, $\gamma = 1$, and the constant advective field $\boldsymbol{\beta} = (1, 1, 1)^T$, the divergence of the velocity field is strictly zero ($\nabla \cdot \boldsymbol{\beta} = 0$). Consequently, the differential operator simplifies to:

$$-\Delta u + \boldsymbol{\beta} \cdot \nabla u + u = f \quad \text{in } \Omega. \tag{19}$$

By applying this operator to our manufactured solution $u_{ex}(x, y, z) = \sin(\pi x)\sin(\pi y)\sin(\pi z)$, we can evaluate the individual contributions. The diffusive term consists of the Laplacian of u_{ex}, which simply scales the solution by a factor of $-3\pi^2$:

$$-\Delta u_{ex} = -\left(\frac{\partial^2}{\partial x^2} + \frac{\partial^2}{\partial y^2} + \frac{\partial^2}{\partial z^2}\right) u_{ex} = 3\pi^2 \sin(\pi x)\sin(\pi y)\sin(\pi z). \tag{20}$$

The advective term requires the evaluation of the directional derivative along $\boldsymbol{\beta}$, resulting in a sum of first-order partial derivatives:

$$\boldsymbol{\beta} \cdot \nabla u_{ex} = \frac{\partial u_{ex}}{\partial x} + \frac{\partial u_{ex}}{\partial y} + \frac{\partial u_{ex}}{\partial z}, \tag{21}$$

where each derivative transforms a sine function into a cosine in the respective spatial direction, multiplied by the chain-rule factor π.
Summing the diffusive contribution, the advective directional derivative, and the reactive term ($\gamma u_{ex} = u_{ex}$), we finally obtain the analytical forcing function required to balance the equation:

$$\begin{aligned} f(x, y, z) = {} & (3\pi^2 + 1)\sin(\pi x)\sin(\pi y)\sin(\pi z) \\ & + \pi\cos(\pi x)\sin(\pi y)\sin(\pi z) \\ & + \pi\sin(\pi x)\cos(\pi y)\sin(\pi z) \\ & + \pi\sin(\pi x)\sin(\pi y)\cos(\pi z). \end{aligned} \tag{22}$$

8.2.2 Qualitative validation

To ensure the reproducibility of the tests and provide a flexible testing environment, the validation executables expose a structured Command Line Interface (CLI). The general execution syntax is the following:

```
mpirun -n <procs> ./<executable> <profiling> <refinement> <mg_flag> <p>
```

where the positional arguments are defined as:

- `<profiling>`: a boolean flag (`0` or `1`). If set to `0`, the solver exports the VTU files for ParaView visualization. If set to `1`, heavy I/O operations are bypassed to perform pure performance profiling and Error Convergence (EOC) analysis.
- `<refinement>`: an integer defining the target maximum level of global mesh refinement. During validation, the solver computes the error norms for all grids from level 2 up to this maximum. In non-profiling runs, the VTU files for visualization are generated solely at this highest refinement level.
- `<mg_flag>`: the string `-mg` enables the geometric multigrid preconditioner, whereas any other string defaults to no preconditioning.
- `<p>`: an integer (from 1 to 12) specifying the polynomial degree p of the continuous Galerkin finite element space V_h.

Note that this general interface is used for any executable in the repository.
For the qualitative validation of the homogeneous problem, we visually inspect the numerical solution. Therefore, we run the solvers on 4 MPI processes, disabling the profiling mode (`0`) to trigger the ParaView output, applying 5 global mesh refinements, and employing a polynomial degree $p = 3$ with the multigrid preconditioner enabled:

```
mpirun -n 4 ./validation_based 0 5 -mg 3
mpirun -n 4 ./validation_free 0 5 -mg 3
```

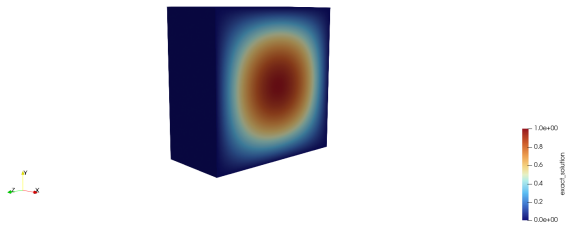

Figure 1: View with ParaView + Clip on plane $x = 0.5$ of the exact solution from Validation_free.cpp

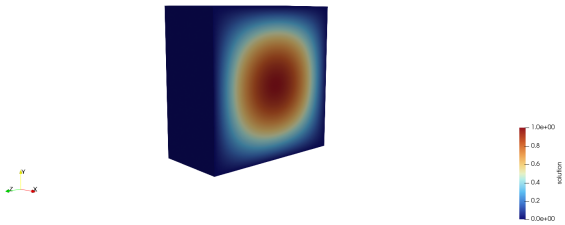

Figure 2: View with ParaView + Clip on plane $x = 0.5$ of the numerical solution from Validation_based.cpp

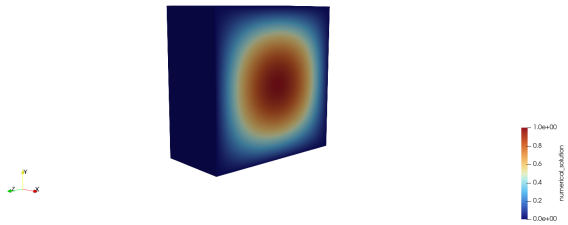

Figure 3: View with ParaView + Clip on plane $x = 0.5$ of the numerical solution from Validation_free.cpp

8.2.3 Error Convergence Analysis

We conclude the verification of this problem by verifying that the convergence properties are aligned with the theoretical results. To achieve this, we halve the mesh size h between subsequent executions by increasing the mesh_refinement_level, as can be seen in both validation_free and validation_based. We compute both the L^2 and H^1 norm of the error and estimate the order of convergence using `deal.II` class `Convergence Table`. We obtained the following results by employing a polynomial degree $p = 6$ and applying 6 global mesh refinements with the multigrid preconditioner enabled:

```
mpirun -n 4 ./validation_based 1 6 -mg 2
mpirun -n 4 ./validation_free 1 6 -mg 2
```

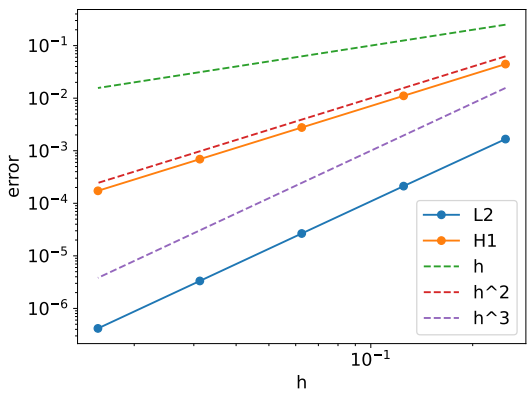

Figure 4: log-log plot of L^2 and H^1 error norm against h (matrix based)

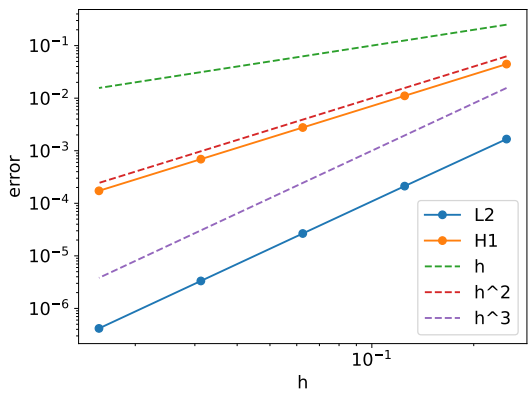

Figure 5: log-log plot of L^2 and H^1 error norm against h (matrix free)

h	L^2 **Error**	**EOC** L^2	H^1 **Error**	**EOC** H^1
0.25	1.67×10^{-3}	-	4.45×10^{-2}	-
0.125	2.12×10^{-4}	2.97	1.11×10^{-2}	2.01
0.0625	2.66×10^{-5}	2.99	2.77×10^{-3}	2.00
0.03125	3.33×10^{-6}	3.00	6.91×10^{-4}	2.00
0.015625	4.16×10^{-7}	3.00	1.73×10^{-4}	2.00

Table 2: Convergence history for the matrix-based solver with polynomial degree $p = 2$ on the homogeneous problem.

h	L^2 **Error**	**EOC** L^2	H^1 **Error**	**EOC** H^1
0.25	1.66×10^{-3}	-	4.44×10^{-2}	-
0.125	2.12×10^{-4}	2.97	1.10×10^{-2}	2.01
0.0625	2.66×10^{-5}	2.99	2.76×10^{-3}	2.00
0.03125	3.33×10^{-6}	3.00	6.91×10^{-4}	2.00
0.015625	4.16×10^{-7}	3.00	1.72×10^{-4}	2.00

Table 3: Convergence history for the matrix-free solver with polynomial degree $p = 2$ on the homogeneous problem.

As illustrated in Figures 4 and 5, the log-log plots of the error norms against the mesh size h exhibit perfectly straight lines, indicating asymptotic algebraic convergence. For a polynomial degree $p = 2$, both solvers precisely match the theoretical asymptotes (dashed lines): the L^2-norm curve is parallel to h^3, yielding a convergence rate of $\mathcal{O}(h^{p+1})$, while the H^1-norm curve is parallel to h^2, yielding a rate of $\mathcal{O}(h^p)$. Furthermore, comparing Table 2 and Table 3, the numerical errors produced by the Matrix-Based and Matrix-Free solvers are identical up to the solver tolerance and the EOCs confirm the theoretical results. This provides strong empirical evidence that the on-the-fly sum-factorization approach is equivalent to the classical sparse matrix assembly, introducing no additional discretization or integration errors.

8.3. Second problem: inhomogeneous boundary conditions

The validation is also necessary to confirm the correctness of the **Hybrid Explicit Integration** approach adopted for the inhomogeneous Dirichlet boundary conditions. Since the matrix-free solver implicitly handles homogeneous systems, the boundary shift must be computed explicitly during the right-hand side assembly. For this purpose, further testing was conducted to compare the matrix-free approach with the standard matrix-based solver. Let us consider the following Diffusion-Reaction problem:

$$\begin{cases} -\nabla \cdot (\mu \nabla u) + \nabla \cdot (\boldsymbol{\beta} u) + \gamma u = f & \text{in } \Omega, \quad (23a) \\ u = g & \text{on } \Gamma_D, \quad (23b) \\ \mu \nabla u \cdot \mathbf{n} = h & \text{on } \Gamma_N. \quad (23c) \end{cases}$$

For this validation step, we chose a diffusion-reaction problem to demonstrate a key architectural feature of our implementation: the dynamic selection of the linear solver and preconditioner. Our code evaluates the advection vector at runtime to determine whether the continuous operator yields a Symmetric Positive Definite (SPD) algebraic system; this triggers the SPD-optimized execution path (employing the Conjugate Gradient method) and allows us to validate both operational modes of our solver across the two test cases. The chosen

parameters and the corresponding manufactured solution are:

$$\Omega = [0,1]^3 \subset R^3$$

$$\Gamma_D = \{x=0\} \cup \{y=0\} \cup \{z=0\}$$
$$\Gamma_N = \{x=1\} \cup \{y=1\} \cup \{z=1\}$$

$$\mu = 1, \quad \boldsymbol{\beta} = (0,0,0)^T, \quad \gamma = 1$$

$$u_{ex}(x,y,z) = e^{x+y+z}$$

$$f(x,y,z) = -2e^{x+y+z}$$

$$g(x,y,z) = e^{x+y+z} \qquad \text{on } \Gamma_D$$

$$h(x,y,z) = e^{x+y+z} \qquad \text{on } \Gamma_N$$

8.3.1 Manufactured forcing function derivation

Following the Method of Manufactured Solutions, we analytically derive the forcing term f and the boundary data g and h by enforcing the exact solution $u_{ex}(x,y,z) = e^{x+y+z}$ into the strong form of the problem, as in the first problem.
Given $\boldsymbol{\beta} = \mathbf{0}$, the differential operator reduces to the symmetric diffusion-reaction equation:

$$-\Delta u + u = f \quad \text{in } \Omega. \tag{24}$$

We evaluate the action of the operator on the exponential solution. The Laplacian operator preserves the exponential function up to a constant factor:

$$-\Delta u_{ex} = -\left(\frac{\partial^2}{\partial x^2} + \frac{\partial^2}{\partial y^2} + \frac{\partial^2}{\partial z^2}\right) e^{x+y+z} = -3e^{x+y+z}. \tag{25}$$

Adding the reactive term ($\gamma u_{ex} = 1 \cdot e^{x+y+z}$), the total analytical forcing function becomes:

$$f(x,y,z) = -3e^{x+y+z} + e^{x+y+z} = -2e^{x+y+z}. \tag{26}$$

The non-homogeneous Dirichlet data g is trivially obtained by evaluating u_{ex} on the boundaries Γ_D. For the Neumann data h, we must compute the normal derivative on Γ_N. The gradient of the exact solution is $\nabla u_{ex} = (e^{x+y+z}, e^{x+y+z}, e^{x+y+z})^T$. Since the Neumann partition consists of the upper faces of the unit cube ($x=1$, $y=1$, $z=1$), the outward unit normal vectors $\mathbf{n}$ are precisely the standard basis vectors. Therefore, the dot product $\nabla u_{ex} \cdot \mathbf{n}$ extracts the function itself along those faces, yielding $h = e^{x+y+z}$.
Despite this coincidental equivalence, the object-oriented implementation of the boundary evaluator avoids hardcoding this scalar result. Instead, it explicitly constructs the exact gradient tensor and computes its inner product with the geometric outward normal $\mathbf{n}$ evaluated on the fly. This guarantees that the software architecture remains robust, general, and mathematically rigorous, automatically ensuring the correct physical sign even if the boundary IDs were re-partitioned (e.g., evaluating a strictly negative flux on the lower faces located at the origin).

8.3.2 Qualitative validation

These results are obtained by running:

```
mpirun -n 4 ./validation_based 0 5 -mg 3
mpirun -n 4 ./validation_free 0 5 -mg 3
```

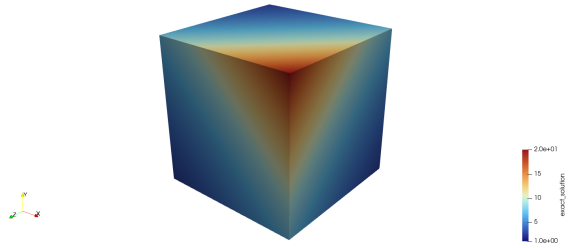

Figure 6: View with ParaView of the exact solution from Validation_inhomogeneous_free.cpp

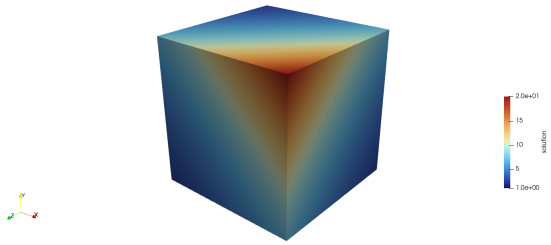

Figure 7: View with ParaView of the numerical solution from Validation_inhomogeneous_based.cpp

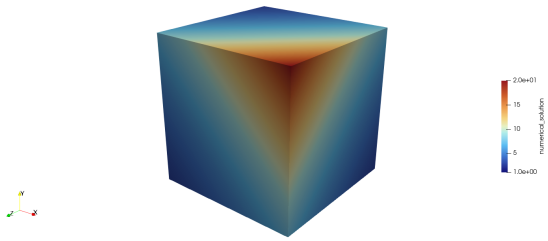

Figure 8: View with ParaView of the numerical solution from Validation_inhomogeneous_free.cpp

8.3.3 Error Convergence Analysis

We proceed by following the same approach used in the homogeneous problem validation. The same considerations made for the homogeneous problem hold. We obtained the following results by running:

```
mpirun -n 4 ./validation_inhomogeneous_based 1 6 -mg 2
mpirun -n 4 ./validation_inhomogeneous_free 1 6 -mg 2
```

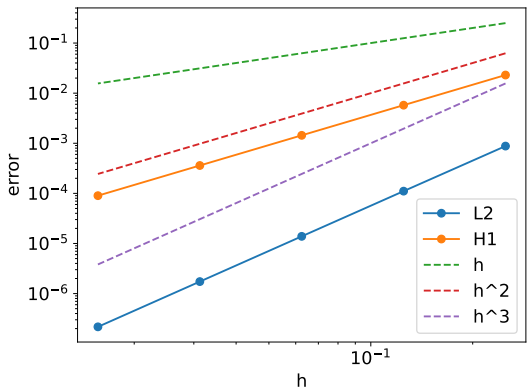

Figure 9: log-log plot of L^2 and H^1 error norm against h (matrix based)

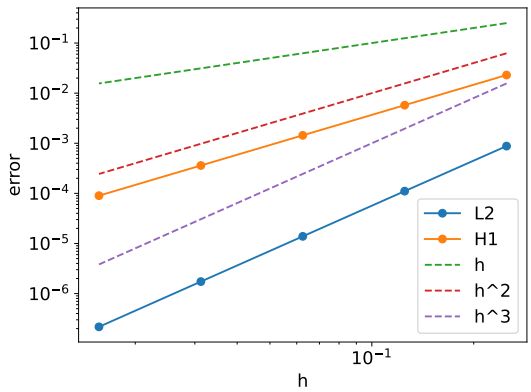

Figure 10: log-log plot of L^2 and H^1 error norm against h (matrix free)

h	L^2 **Error**	**EOC** L^2	H^1 **Error**	**EOC** H^1
0.25	8.79×10^{-4}	-	2.30×10^{-2}	-
0.125	1.11×10^{-4}	2.99	5.76×10^{-3}	2.00
0.0625	1.39×10^{-5}	3.00	1.44×10^{-3}	2.00
0.03125	1.74×10^{-6}	3.00	3.60×10^{-4}	2.00
0.015625	2.17×10^{-7}	3.00	9.00×10^{-5}	2.00

Table 4: Convergence history for the matrix-based solver with polynomial degree $p = 2$ on the inhomogeneous problem.

h	L^2 **Error**	**EOC** L^2	H^1 **Error**	**EOC** H^1
0.25	8.78×10^{-4}	-	2.30×10^{-2}	-
0.125	1.11×10^{-4}	2.99	5.76×10^{-3}	2.00
0.0625	1.39×10^{-5}	3.00	1.44×10^{-3}	2.00
0.03125	1.74×10^{-6}	3.00	3.60×10^{-4}	2.00
0.015625	2.17×10^{-7}	3.00	9.00×10^{-5}	2.00

Table 5: Convergence history for the matrix-free solver with polynomial degree $p = 2$ on the inhomogeneous problem.

Remark on the standalone Matrix-Free solver: For the sake of completeness, it is worth mentioning that the entire error convergence analysis, for both the homogeneous and inhomogeneous problems, has also been successfully conducted for the Matrix-Free solver without the Multigrid preconditioner feature. Since the computed errors are practically indistinguishable from those obtained with the preconditioned solver and the EOCs strictly match the theoretical asymptotes, the corresponding plots and tables have been omitted for the sake of brevity.

9. Profiling

9.1. Problem definition

To systematically evaluate the computational performance and scalability of the solvers, we define a standard three-dimensional Advection-Diffusion-Reaction (ADR) problem. Unlike the verification phase, where the forcing term and boundary data were mathematically manufactured to match a known analytical solution, the profiling benchmark employs simple constant scalar fields.
The problem is formulated as usual:

$$\begin{cases} -\nabla \cdot (\mu \nabla u) + \nabla \cdot (\boldsymbol{\beta} u) + \gamma u = f & \text{in } \Omega, \quad (27a) \\ u = g & \text{on } \Gamma_D, \quad (27b) \\ \mu \nabla u \cdot \mathbf{n} = h & \text{on } \Gamma_N. \quad (27c) \end{cases}$$

The spatial domain Ω is the unit hypercube. Following the standard `deal.II` face numbering convention (where faces 0 and 1 are orthogonal to the x-axis, 2 and 3 to the y-axis, and 4 and 5 to the z-axis), we partition the boundary by imposing Neumann conditions on the faces $x = 0$ and $z = 1$, while applying Dirichlet constraints on all the remaining faces. The complete set of physical and numerical parameters used for the profiling runs

is:

$$
\begin{aligned}
\Omega &= [0,1]^3 \subset R^3 \\
\Gamma_D &= \bigcup_{i \in \{1,2,3,4\}} \Gamma_i \\
\Gamma_N &= \Gamma_0 \cup \Gamma_5 \\
\mu &= 3.0, \quad \boldsymbol{\beta} = (1.0, -1.0, 1.0)^T, \quad \gamma = 2.0 \\
f(\mathbf{x}) &= 1.0 \\
g(\mathbf{x}) &= 7.0 \qquad \text{on } \Gamma_D \\
h(\mathbf{x}) &= 2.0 \qquad \text{on } \Gamma_N
\end{aligned}
$$

9.2. Hardware Profiling and Performance Metrics

Before evaluating the parallel scalability of the solvers on the HPC cluster, it is fundamental to understand their interaction with the underlying hardware architecture. In standard shared HPC environments (as the Cluster of the Department of Mathematics we used for scaling tests), access to low-level Hardware Performance Counters (e.g., via the Linux **perf** subsystem) is strictly restricted due to security and privilege constraints. Consequently, to gather low-level architectural metrics, we had to perform such measures on a local machine, with strong hardware limitations.
The tests were executed on a workstation equipped with an AMD Ryzen 7 8845HS processor, whose detailed hardware specifications are summarized in Table 6. This architecture features a 16 MiB Last Level Cache (L3) and native support for AVX-512 instruction sets; this provides an optimal testing ground to evaluate the heavily vectorized **FEEvaluation** loops and the memory-bound nature of the standard sparse-matrix assembly.
The profiling workflow was entirely automated using a custom **.sh** bash script. This script iterated over the polynomial degrees $p \in [2, 6]$ at a fixed mesh refinement level ($L = 4$), and invoked both the **perf stat** and **time** utilities to intercept hardware events (such as CPU cycles, L1 data cache accesses, and Last Level Cache misses) directly from the processor registers and parse them into a **.csv** file.

Local Profiling Workstation Specifications	
Processor Model	AMD Ryzen 7 8845HS
Architecture	x86_64 (Zen 4)
Cores / Threads	8 Physical Cores / 16 Logical Threads
Max Boost Clock	Up to 5.1 GHz
L1 Cache	256 KiB Data + 256 KiB Instruction (8 instances)
L2 Cache	8 MiB (8 instances)
L3 Cache (LLC)	16 MiB (Shared)
SIMD Extension	AVX-512
Physical Memory	16 GB

Table 6: Hardware specifications of the local machine used for low-level architectural profiling.

Memory Footprint Analysis

Figure 11 illustrates the peak memory usage of the two solvers. The Matrix-Based approach explicitly allocates the global sparse matrix; as the polynomial degree increases, the dense local cell matrices grow as $\mathcal{O}(p^{2d})$, leading to a severe memory blow-up. At $p = 6$, the matrix-based solver allocates nearly 12 GB, dangerously approaching the physical RAM limit of the machine. Conversely, the Matrix-Free implementation only requires the storage of the solution and right-hand side vectors, keeping the memory footprint virtually flat and demonstrating its potential for high polynomial orders.

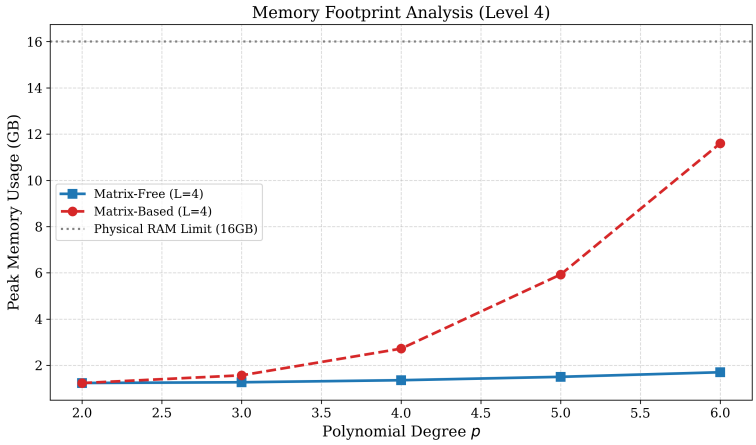

Figure 11: Peak memory usage comparison against the physical hardware limits.

Computational Complexity

The algorithmic complexity is captured in Figure 12, which plots the total CPU cycles on a logarithmic scale. While both solvers exhibit increasing costs as p grows, the Matrix-Free solver is systematically orders of magnitude cheaper. This perfectly reflects the theoretical bounds: substituting the $\mathcal{O}(p^{2d})$ dense matrix-vector product with the $\mathcal{O}(d \cdot p^{d+1})$ Sum-Factorization approach reduces the required floating-point operations and the associated processor cycles.

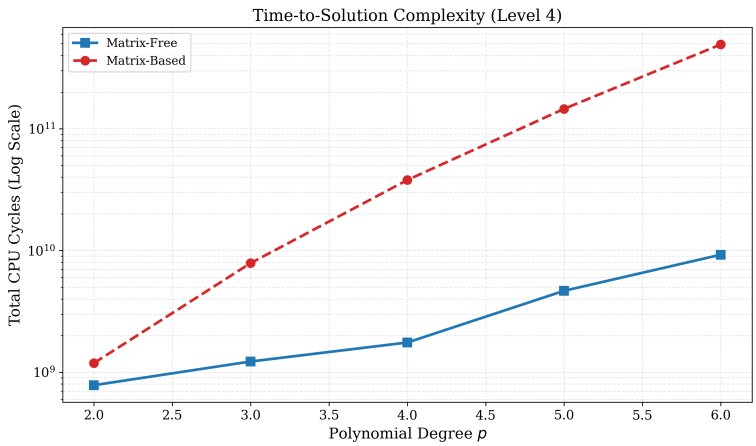

Figure 12: Total CPU cycles (log scale) required to reach convergence.

Cache Behavior and Arithmetic Intensity

A fundamental architectural difference lies in memory hierarchy utilization. Figure 13 normalizes the Last Level Cache (LLC) misses by the total number of Degrees of Freedom. The Matrix-Based solver shows an increasing trend of LLC misses per DoF, symptomatic of a heavily memory-bound application that constantly accesses large matrix rows from the main RAM. The Matrix-Free curve tends to decrease with p. This highlights high arithmetic intensity: as the polynomial degree increases, the solver performs significantly more operations on the data already residing in the L1 cache, minimizing accesses to the slow main memory.
This phenomenon affects the cache miss rates as well, detailed in Figure 14 for $p = 6$. The Matrix-Free approach halves the L1 cache miss rate compared to the Matrix-Based solver (4.57% vs. 8.55%), indicating improved locality of the vectorized FEEvaluation loops. While the Matrix-Free LLC miss rate appears nominally higher (2.05% vs. 0.41%), this is a statistical artifact: because the absolute number of cache references in the Matrix-Free solver is lower by orders of magnitude, the few unavoidable memory fetches constitute a larger percentage of the total, despite less data movement is performed.

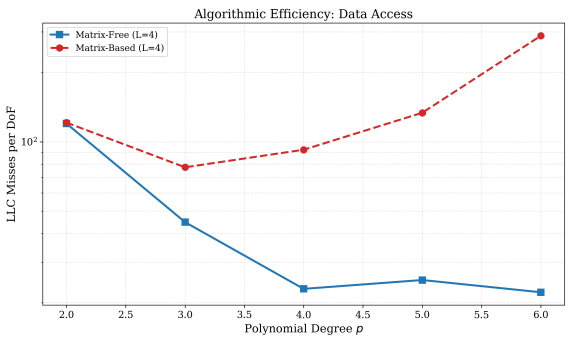

Figure 13: Absolute Last Level Cache (L3) misses normalized by the number of DoFs.

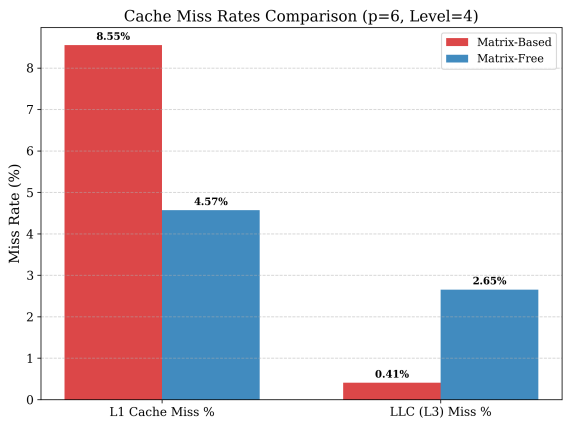

Figure 14: L1 and LLC relative miss rates for polynomial degree $p = 6$.

9.3. Scalability

In this section, all numerical experiments were carried out using the high-performance computing cluster provided by the Department of Mathematics at Politecnico di Milano. The computational infrastructure allowed us to perform large-scale simulations and systematic performance analyses, including strong scaling studies. To avoid thread contention and oversubscription penalties in our pure-MPI scaling benchmarks, the internal task-based parallelism of the `MatrixFree` loop was explicitly disabled via the `AdditionalData::none` flag. This ensured a strict 1-to-1 mapping between MPI processes and physical CPU cores. Moreover, we deliberately restricted the execution to physical cores only, avoiding Hyperthreading to prevent resource contention that could have worsened the performance in this particular application. All performance comparisons were conducted using identical meshes, solver tolerances, and multigrid configurations.

Server Profiling Platform Specifications	
Processor Model	Intel Xeon Gold 6238R @ 2.20 GHz
Architecture	x86_64 (Cascade Lake)
Sockets / Cores / Threads	2 Sockets / 56 Cores / 112 Threads
Base Frequency	2.20 GHz
L1 Cache	1.8 MiB Data + 1.8 MiB Instruction (56 instances)
L2 Cache	56 MiB (56 instances)
L3 Cache (LLC)	77 MiB (2 Shared)
SIMD Extension	AVX, AVX2, AVX-512 (F, DQ, CD, BW, VL, VNNI)
NUMA Nodes	2
Virtualization	Intel VT-x

Table 7: Hardware specifications of the server platform used for large-scale profiling experiments.

To guarantee performance reproducibility and avoid external interference, all benchmarks were submitted to a dedicated queue providing exclusive node access, subject to a strict 30-minute wall-time limit per job.
The profiling strategy was constrained by the limited amount of computational resources and wall-time available for each job on the cluster. As a consequence, some additional configurations could not be included, since longer runs or higher-resolution studies would have exceeded the allocated runtime limits.
Figure 15 shows the setup time and solve time of a Matrix-Free method as the polynomial degree increases from $p = 1$ to $p = 2$. The values are displayed on a logarithmic time scale (milliseconds). Each bar is stacked: the lighter lower portion represents the setup phase, while the darker upper portion represents the solve phase. Overall, the total runtime grows rapidly with increasing degree. The solve stage dominates the cost at all degrees and becomes increasingly significant at higher orders, while the setup time grows more moderately.

Figure 16 presents the relative composition of runtime, showing the fraction of total time spent in setup versus solve for each degree. The bars are normalized to one, so they represent percentages instead of absolute time. At low polynomial degrees (p =1-3), the setup phase occupies a noticeable portion of the runtime, whereas at higher degrees the solve phase exceeds about 90% of the total time, making the setup overhead almost negligible.

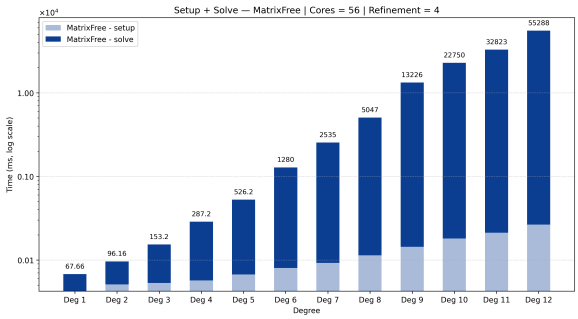

Figure 15: Absolute setup and solve times (Matrix-Free).

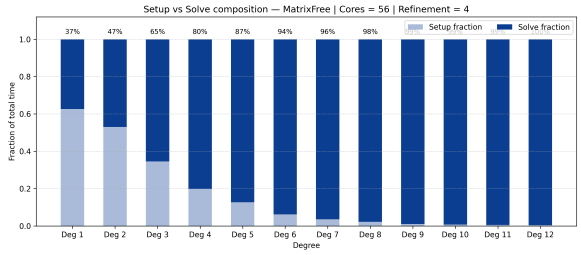

Figure 16: Runtime composition fractions (Matrix-Free).

Figure 17 compares the setup and solve times of the *Matrix-Based* and *Matrix-Free* approaches for increasing polynomial degrees on 56 cores. The Matrix-Based method exhibits significantly larger total runtimes, with both setup and solve costs growing rapidly, whereas the Matrix-Free approach remains consistently cheaper and scales more favorably. The performance gap widens at higher polynomial degrees.
Figure 18 shows the fractional composition of runtime. The Matrix-Based method maintains a substantial setup fraction, reflecting increasing assembly costs, while the Matrix-Free implementation is dominated by the solve phase and becomes almost entirely solve-bound at higher degrees.

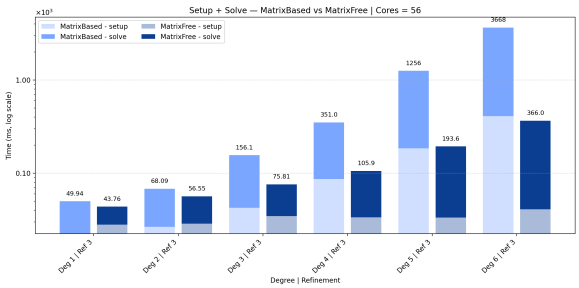

Figure 17: Absolute runtime comparison.

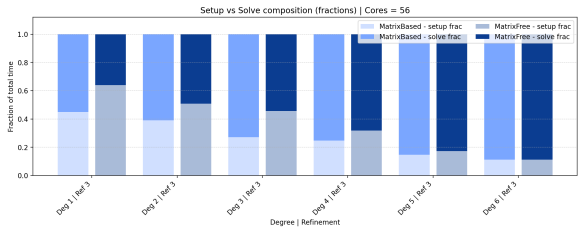

Figure 18: Runtime composition comparison.

Strong scalability

Figure 19 shows the strong scaling speedup and efficiency for Degree 12 with Refinement 6 as the number of cores increases. The dashed line represents the ideal linear speedup, while the solid curves correspond to the Matrix-Free and Matrix-Based implementations. Both approaches follow the expected scaling trend at low core counts but gradually deviate from ideal behavior due to increasing parallel overhead. The Matrix-Based method shows limited speedup and poorer scalability. However, more relevant metrics are shown in the next section.

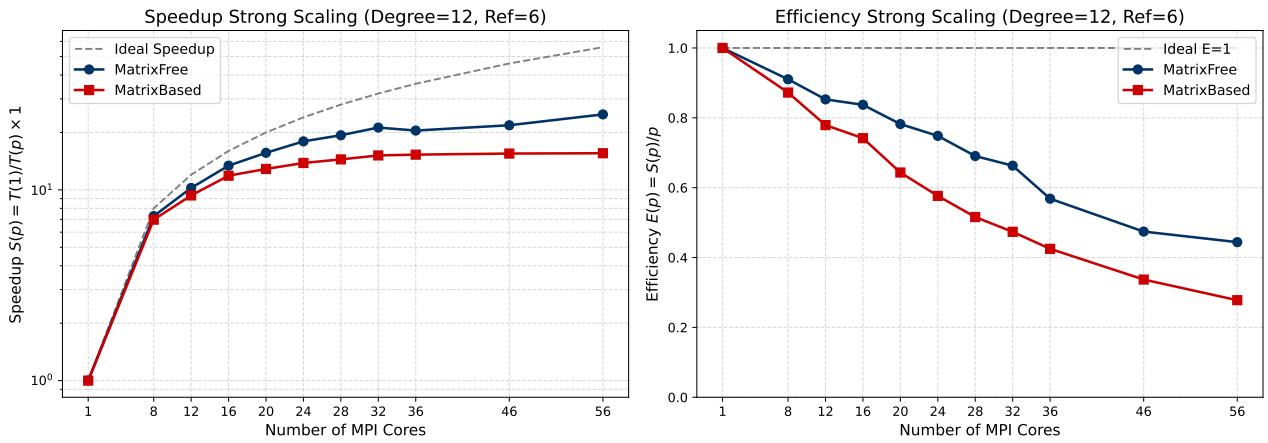

Figure 19: Strong scaling speedup and efficiency plot .

Throughput analysis

While evaluating the computational efficiency of a PDE solver on modern hardware, relative strong scaling speedup can sometimes be a deceptive metric, as it often masks an inefficient baseline performance. A rigorous assessment requires analyzing absolute metrics: the total execution time and the computational throughput, measured in millions of Degrees of Freedom processed per second (MDoFs/s) during the iterative solution phase. Figure 20 illustrates these absolute performance indicators for a high-order scenario ($p = 12$).

The left panel, presented in a logarithmic scale, highlights a massive gap in the total wall-time. The Matrix-Based solver starts from a severely penalized baseline due to the prohibitive memory footprint and computational cost required to assemble and solve the global sparse matrix, remaining roughly an order of magnitude slower than its Matrix-Free counterpart across all core counts.

The right panel reveals the hardware-level root cause of this performance discrepancy. The Matrix-Based solver's throughput hits a hard plateau early in the parallel scaling process, stalling near 17 MDoFs/s. Adding more processing units yields no performance benefit because the dense $\mathcal{O}(p^{2d})$ local matrices and the massive global sparse structure completely saturate the main memory bandwidth. Consequently, the CPU cores are forced to stall while waiting for data fetches from the RAM—a textbook manifestation of the *memory wall*.

On the other hand, the Matrix-Free solver proofs its architectural superiority, effectively scaling up to over 60 MDoFs/s. By replacing the explicit matrix storage with on-the-fly sum-factorization, the solver drastically increases the arithmetic intensity. This design allows the processing units to fully exploit the AVX-512 vectorization directly within the CPU's cache hierarchy, bypassing the memory bus bottleneck.

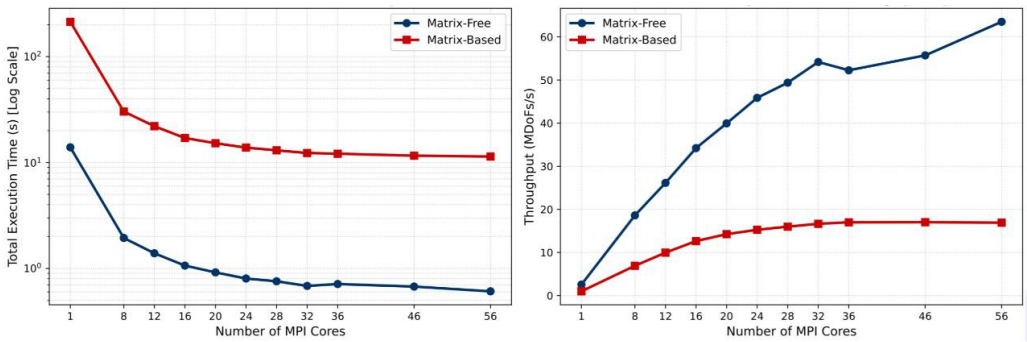

Figure 20: Comparison of the Total Execution Time (s) and Computational Throughput (MDoFs/s) between the Matrix-Free and Matrix-Based solvers ($p = 12$,ref= 6).

Weak scalability

To evaluate weak scalability, we opted for a two-dimensional problem domain ($d = 2$). The choice of $d = 2$ over a three-dimensional setup ($d = 3$) was strictly driven by hardware constraints: scaling a 3D problem with increasing mesh refinement to maintain a constant workload per core would have required a total number of cores far exceeding the availability of the Politecnico HPC cluster. Nevertheless, we performed some tests exploiting the maximum computational power available (limited to 56 physical cores). Despite the restricted number of data points, the experiment successfully isolates the impact of network overhead, which suggests architectural robustness and scalability of the matrix-free approach compared to classical matrix assembly on distributed-memory systems.
In configuring the weak scaling benchmark, the workload balance was calibrated based on the number of mesh elements rather than the global Degrees of Freedom (DoFs). By incrementing the refinement level, the total number of active cells exactly quadrupled at each step, ensuring that every MPI process handled a constant number of elements (precisely 16,384 cells per core across the 1, 4, and 16-core configurations).
The total number of DoFs does not scale with the same precision (i.e., with a deviation of less than 0.2%) due to shared nodes at the boundaries, but the constant-load assumption remains mathematically rigorous. The `MatrixFree` framework in `deal.II` evaluates the operator action by strictly iterating cell by cell. Consequently, keeping the number of cells per process constant guarantees that the number of floating-point operations remains invariant for each CPU thread.
As shown in Figure 21, the Matrix-Free solver maintains a stable execution time. Conversely, the total wall-time of the Matrix-Based solver increases as the core count grows. This performance degradation can be attributed to the bottleneck introduced by the distributed sparse matrix assembly and the memory bandwidth saturation.

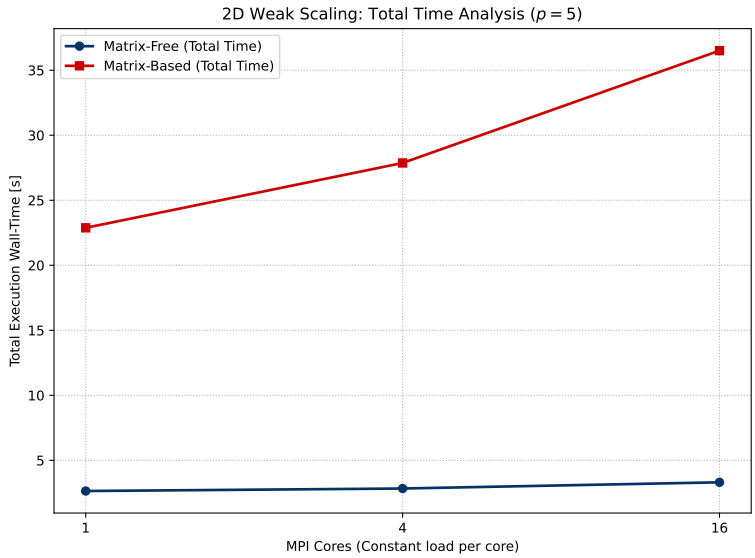

Figure 21: Total execution wall-time for the 2D weak scaling benchmark at ($p = 5$). The problem size is scaled proportionally to the number of active MPI processes.

10. Conclusions

The results highlight a clear trade-off between implementation complexity and computational efficiency. While the assembled approach offers a simpler development workflow, its performance becomes increasingly limited by memory bandwidth at higher polynomial degrees.
In contrast, the matrix-free formulation requires a more sophisticated implementation but provides superior arithmetic intensity, reduced memory footprint, and improved scalability. These characteristics make it particularly suitable for high-order discretizations on modern multi-core architectures.

References

[1] P. C. Africa, M. Salvador, P. Gervasio, L. Dedè, and A. Quarteroni. A matrix-free high-order solver for the numerical solution of cardiac electrophysiology. *Journal of Computational Physics*, 478:111984, 2023.

[2] deal.II Development Team. The deal.ii tutorial: Step-37, 2024.

[3] deal.II Development Team. Matrix-free infrastructure / representation of inhomogeneous dirichlet boundary conditions for continuous finite elements, 2024.

[4] David B. Kirk and Wen-mei W. Hwu. *Programming Massively Parallel Processors: A Hands-on Approach*. Morgan Kaufmann, 2nd edition, 2012.

[5] Martin Kronbichler. The state of matrix-free methods and hpc. Presentation at the Eighth deal.II Users and Developers Workshop, May 2020. Technical University of Munich, Germany.

[6] Martin Kronbichler and Katharina Kormann. A generic interface for parallel cell-based finite element operator application. *Computers & Fluids*, 63:135–147, 2012.

[7] Peter Pacheco. *An Introduction to Parallel Programming*. Morgan Kaufmann, 2011.

[8] Alfio Quarteroni. *Numerical Models for Differential Problems*. Springer, 3rd edition, 2017.